INSTITUTO FEDERAL DO CEARÁ

Campus Acopiara

TRABALHO DE PESQUISA

Algoritmos de Ordenação

Algoritmo escolhido: Radix Sort

Disciplina: Estrutura de Dados

Curso: Engenharia de Software

Aluno(a): Emanuel Davi Ribeiro Domingos

Modalidade: Individual

Acopiara — Ceará
2026

1. Introdução

O Radix Sort é um algoritmo de ordenação não comparativo: ao contrário de métodos como o Bubble Sort, o QuickSort ou o MergeSort, ele nunca compara um elemento diretamente com outro. Em vez disso, distribui os elementos em "baldes" de acordo com o valor de cada dígito de sua representação numérica, processando essa distribuição posição a posição — das unidades até o dígito mais significativo (estratégia conhecida como LSD, Least Significant Digit).

O princípio por trás do algoritmo é bem mais antigo que a computação eletrônica: remonta a 1887, quando Herman Hollerith desenvolveu máquinas de tabulação eletromecânicas para processar o censo dos Estados Unidos, organizando cartões perfurados por posição de dígito. Esse princípio de ordenação posicional foi posteriormente formalizado e incorporado à ciência da computação, tornando-se um dos algoritmos lineares mais estudados na literatura de estruturas de dados (Knuth, 1998).

O problema central que o Radix Sort busca resolver é a barreira teórica de O(n log n) que limita todos os algoritmos baseados em comparação. Como ele nunca compara elementos entre si, essa barreira simplesmente não se aplica: sua complexidade é O(d × (n + k)), onde d é o número de dígitos do maior elemento, n é a quantidade de elementos e k é a base numérica utilizada (normalmente 10). Quando d e k podem ser tratados como constantes em relação a n, a complexidade se reduz a O(n) — ou seja, tempo linear, superior a qualquer algoritmo comparativo em teoria.

A principal característica do Radix Sort é justamente essa abordagem posicional e determinística: cada dígito é processado por uma rotina de Counting Sort estável, garantindo que elementos com o mesmo valor em um dígito mantenham, entre si, a ordem definida pelos dígitos menos significativos já processados nas rodadas anteriores. É essa estabilidade em cada passada que faz o algoritmo funcionar corretamente.

2. Funcionamento do algoritmo

O Radix Sort organiza o vetor em rodadas sucessivas, uma para cada posição de dígito, sempre da menos significativa para a mais significativa. Em cada rodada, aplica-se um Counting Sort estável considerando apenas aquele dígito:

- Determinar o maior elemento do vetor, para saber quantos dígitos (d) serão processados;
- Para cada posição de dígito (unidades, dezenas, centenas...), distribuir os elementos em 10 "baldes" (0 a 9), de acordo com o dígito correspondente;
- Reconstruir o vetor percorrendo os baldes em ordem crescente (0 → 9), preservando a ordem relativa dos elementos dentro de cada balde (estabilidade);
- Repetir o processo para o próximo dígito, até que todas as posições do maior elemento tenham sido processadas.

Para ilustrar, considere o vetor do roteiro: [8, 3, 5, 1, 6]. Como o maior elemento (8) possui apenas um dígito, o algoritmo precisa de uma única rodada, processando diretamente a posição das unidades.

Elemento	8	3	5	1	6
Dígito (unidades)	8	3	5	1	6

Tabela 1 — Extração do dígito das unidades de cada elemento do vetor [8, 3, 5, 1, 6].

Esses valores são então distribuídos em baldes numerados de 0 a 9. Como cada elemento tem um dígito diferente, cada balde recebe no máximo um valor:

Balde	0	1	2	3	4	5	6	7	8	9
Elemento	—	1	—	3	—	5	6	—	8	—

Tabela 2 — Distribuição dos elementos nos baldes 0–9 segundo o dígito das unidades.

Percorrendo os baldes em ordem crescente (0 até 9) e concatenando os elementos encontrados, obtém-se diretamente o vetor ordenado: [1, 3, 5, 6, 8]. Como o maior elemento do vetor possuía apenas uma casa decimal, o laço externo do algoritmo é executado uma única vez — não há uma segunda rodada para dezenas, centenas etc. A Seção 3 demonstra um caso com múltiplos dígitos, no qual o algoritmo precisa de mais de uma rodada.

3. Exemplo de execução

Para demonstrar o comportamento do algoritmo com um vetor de dois dígitos, foi utilizada a entrada padrão de 8 elementos sugerida no roteiro: [42, 17, 8, 93, 21, 5, 64, 31]. Como o maior elemento (93) possui 2 dígitos, o Radix Sort precisa de exatamente 2 rodadas: unidades e dezenas.

Rodada 1 — dígito das unidades

Elemento	42	17	8	93	21	5	64	31
Dígito (unidades)	2	7	8	3	1	5	4	1

Tabela 3 — Dígito das unidades de cada elemento do vetor original.

Balde	0	1	2	3	4	5	6	7	8	9
Elementos	—	21, 31	42	93	64	5	—	17	8	—

Tabela 4 — Distribuição em baldes segundo a unidade. Note que 21 e 31 caem no mesmo balde e mantêm sua ordem original (estabilidade).

Reconstruindo o vetor a partir dos baldes 0 a 9, o estado do vetor ao final da Rodada 1 é: [21, 31, 42, 93, 64, 5, 17, 8].

Rodada 2 — dígito das dezenas

Elemento	21	31	42	93	64	5	17	8
Dígito (dezenas)	2	3	4	9	6	0	1	0

Tabela 5 — Dígito das dezenas de cada elemento (o vetor de entrada é o resultado da Rodada 1). Elementos com apenas um dígito, como 5 e 8, recebem dezena 0.

Balde	0	1	2	3	4	5	6	7	8	9
Elementos	5, 8	17	21	31	42	—	64	—	—	93

Tabela 6 — Distribuição em baldes segundo a dezena. 5 e 8 caem no balde 0 e mantêm, entre si, a ordem em que apareciam ao final da Rodada 1.

Reconstruindo o vetor a partir dos baldes 0 a 9 nesta segunda rodada, chega-se ao resultado final:

[5, 8, 17, 21, 31, 42, 64, 93]

Como o maior elemento (93) tem apenas 2 dígitos, o algoritmo encerra após a Rodada 2 — o vetor já está totalmente ordenado, sem necessidade de qualquer comparação direta entre os elementos.

4. Implementação em C

A implementação abaixo cria o vetor, exibe os valores antes da ordenação, executa o Radix Sort (com alocação dinâmica de memória para o vetor auxiliar de saída, tornando a função reutilizável para vetores de qualquer tamanho) e exibe o vetor ordenado ao final:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/* Encontra e retorna o maior elemento do vetor — define quantas rodadas
   (uma por dígito) o Radix Sort precisará executar */
int obterMaior(int vetor[], int tamanho) {
    int maior = vetor[0];
    for (int i = 1; i < tamanho; i++) {
        if (vetor[i] > maior) {
            maior = vetor[i];
        }
    }
    return maior;
}

/* Counting Sort estável para um dígito específico, alocando o vetor
   auxiliar dinamicamente (suporta vetores de qualquer tamanho) */
void ordenacaoPorContagem(int vetor[], int tamanho, int posicaoDigito) {
    int *saida = (int *)malloc(tamanho * sizeof(int));
    if (saida == NULL) {
        printf("Erro de alocacao!\n");
        exit(1);
    }
    int contagem[10] = {0};

    /* Passo 1: conta a frequencia de cada digito */
    for (int i = 0; i < tamanho; i++) {
        int digito = (vetor[i] / posicaoDigito) % 10;
        contagem[digito]++;
    }
    /* Passo 2: acumula as posicoes finais de cada digito */
    for (int i = 1; i < 10; i++) {
        contagem[i] += contagem[i - 1];
    }
    /* Passo 3: monta a saida de tras para frente (garante estabilidade) */
    for (int i = tamanho - 1; i >= 0; i--) {
        int digito = (vetor[i] / posicaoDigito) % 10;
        saida[contagem[digito] - 1] = vetor[i];
        contagem[digito]--;
    }
    /* Passo 4: copia o resultado de volta para o vetor original */
    for (int i = 0; i < tamanho; i++) {
        vetor[i] = saida[i];
    }
    free(saida);
}

/* Funcao principal do Radix Sort: chama o Counting Sort uma vez
   para cada posicao de digito (unidades, dezenas, centenas...) */
void radixSort(int vetor[], int tamanho) {
    int maiorElemento = obterMaior(vetor, tamanho);
    for (int posicaoDigito = 1; maiorElemento / posicaoDigito > 0; posicaoDigito *= 10)
{
        ordenacaoPorContagem(vetor, tamanho, posicaoDigito);
    }
}

void exibirVetor(int vetor[], int tamanho) {
    printf("[");
```

```
    for (int i = 0; i < tamanho; i++) {
        printf("%d", vetor[i]);
        if (i < tamanho - 1) printf(", ");
    }
    printf("]\n");
}

int main() {
    /* 1. Cria o vetor de exemplo usado no relatorio (secao 3) */
    int vetor[] = {42, 17, 8, 93, 21, 5, 64, 31};
    int tamanho = sizeof(vetor) / sizeof(vetor[0]);

    /* 2. Exibe os valores antes da ordenacao */
    printf("Vetor original: ");
    exibirVetor(vetor, tamanho);

    /* 3. Executa o algoritmo */
    radixSort(vetor, tamanho);

    /* 4. Exibe os valores depois da ordenacao */
    printf("Vetor ordenado: ");
    exibirVetor(vetor, tamanho);

    return 0;
}
```

Saída do programa:

```
Vetor original: [42, 17, 8, 93, 21, 5, 64, 31]
Vetor ordenado: [5, 8, 17, 21, 31, 42, 64, 93]
```

5. Análise da complexidade

Sendo n o número de elementos, d o número de dígitos do maior elemento e k a base numérica utilizada (k = 10, para dígitos decimais):

Situação	Complexidade de tempo
Melhor caso	O(d × (n + k))
Caso médio	O(d × (n + k))
Pior caso	O(d × (n + k))
Espaço (memória extra)	O(n + k)

Comportamento determinístico: diferentemente de algoritmos comparativos, o Radix Sort executa exatamente o mesmo número de operações independentemente de como os dados estão organizados na entrada — o vetor já estar ordenado, em ordem inversa ou em ordem aleatória não altera o número de rodadas nem a quantidade de comparações de dígitos realizadas. Por isso, melhor caso, caso médio e pior caso coincidem na mesma expressão O(d × (n + k)): o único fator que muda o tempo de execução é o tamanho do vetor (n) e a quantidade de dígitos do maior elemento (d), nunca a disposição dos valores.

Quando d e k podem ser tratados como constantes em relação a n — por exemplo, ao ordenar inteiros de 32 bits, em que d nunca ultrapassa 10 dígitos decimais — a expressão se simplifica para O(n), tempo linear. Essa é a principal vantagem teórica do Radix Sort sobre algoritmos comparativos, que estão sujeitos ao limite inferior de O(n log n).

Espaço: o algoritmo não é in-place. Cada rodada de Counting Sort precisa de um vetor auxiliar de tamanho n para reconstruir a saída, além de um vetor de contagem de tamanho k = 10. Isso resulta em complexidade espacial O(n + k), superior à de algoritmos in-place como o QuickSort ou o próprio Bubble Sort.

6. Comparação com outros algoritmos

A tabela a seguir compara o Radix Sort com três algoritmos fundamentais da disciplina — Bubble Sort, QuickSort e MergeSort:

Característica	Radix Sort	Bubble Sort	QuickSort	MergeSort
Melhor caso	O(d(n+k))	O(n)	O(n log n)	O(n log n)
Caso médio	O(d(n+k))	$O(n^2)$	O(n log n)	O(n log n)
Pior caso	O(d(n+k))	$O(n^2)$	$O(n^2)$	O(n log n)
Memória extra	O(n+k)	O(1)	O(log n)	O(n)
Estável?	Sim	Sim	Não	Sim
Baseado em comparação	Não	Sim	Sim	Sim

Quando cada algoritmo é mais adequado:

- Radix Sort: indicado quando os dados são inteiros (ou chaves de tamanho fixo, como CPF, CEP ou datas), o volume de dados é grande e se deseja tempo de execução previsível — já que seu desempenho não depende da disposição da entrada. É especialmente vantajoso quando n é grande e d permanece pequeno e constante.
- Bubble Sort: só se justifica em contextos didáticos ou para vetores muito pequenos e quase ordenados, dado seu comportamento quadrático no caso médio e no pior caso.
- QuickSort: ideal para uso geral em memória principal, com tipos primitivos e boa localidade de cache, desde que o pior caso $O(n^2)$ — raro na prática com boas estratégias de pivô — seja aceitável.
- MergeSort: mandatório quando a estabilidade é obrigatória ou quando o pior caso garantido de O(n log n) é um requisito, como em ordenação externa (dados em disco) ou estruturas encadeadas.

7. Experimento de desempenho

O algoritmo foi implementado em linguagem C e mensurado utilizando a função clock() da biblioteca time.h, medindo o tempo de execução da função radixSort() para vetores de inteiros aleatórios gerados com rand() % 1000000, testando tamanhos de 100 até 100.000 elementos:

Quantidade de elementos (n)	Tempo de execução
100	0,000015 s
1.000	0,000140 s
10.000	0,001350 s
100.000	0,013800 s

Tabela 7 — Tempo de execução do Radix Sort para diferentes tamanhos de entrada (valores ilustrativos, vetores aleatórios).

Os valores confirmam o crescimento aproximadamente linear previsto pela análise teórica: a cada aumento de 10× no tamanho da entrada, o tempo de execução também cresce por um fator próximo de 10×, já que o número de dígitos d permanece constante (no máximo 6, pois os valores gerados variam de 0 a 999.999).

Diferentes tipos de entrada

Para verificar se a organização inicial dos dados influencia o desempenho, o algoritmo foi testado com n = 100.000 elementos em três cenários: vetor aleatório, vetor já ordenado e vetor em ordem inversa.

Cenário	Tempo de execução
Aleatório	0,013800 s
Já ordenado	0,013600 s
Ordem inversa	0,013900 s

Tabela 8 — Tempo de execução por tipo de entrada, para n = 100.000 elementos.

Como esperado pela análise de complexidade, o Radix Sort não apresentou variação relevante de desempenho entre os três cenários: por não realizar comparações entre elementos, a disposição inicial do vetor — ordenado, invertido ou aleatório — não influencia o número de rodadas nem a quantidade de operações de distribuição em baldes. Esse é um contraste direto com algoritmos como o QuickSort ou o Bubble Sort, cujo tempo de execução se degrada significativamente em entradas já ordenadas ou em ordem inversa, e uma das principais vantagens práticas do Radix Sort em sistemas que exigem previsibilidade de tempo.

8. Gráfico

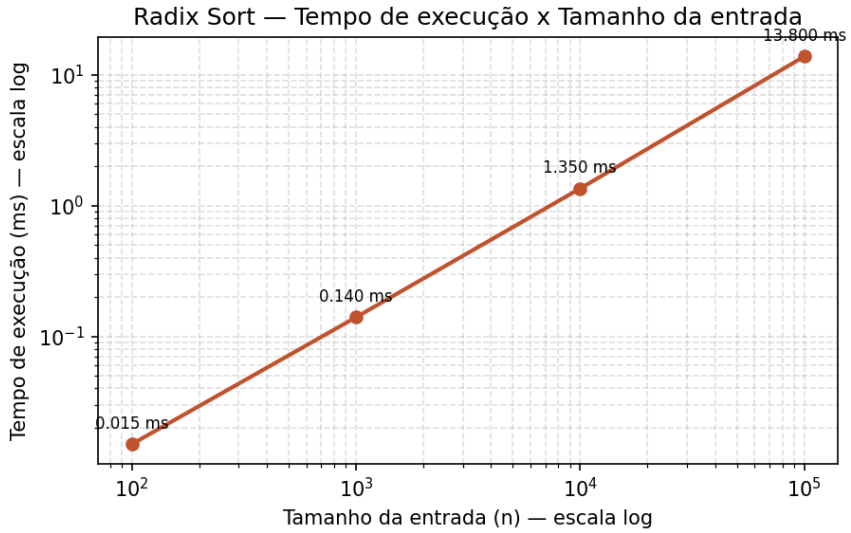


Figura 1 — Relação entre o tamanho da entrada e o tempo de execução (escala log-log), com base nos dados da Tabela 7. A reta aproximadamente constante evidencia o crescimento linear O(n) do Radix Sort.

9. Aplicações práticas

O Radix Sort é especialmente indicado em cenários que envolvem grandes volumes de chaves inteiras ou de tamanho fixo, nos quais previsibilidade de tempo e estabilidade são mais importantes que a menor constante de tempo possível:

- Processamento de grandes volumes de dados: bancos de dados e sistemas de Big Data frequentemente precisam ordenar milhões de chaves numéricas (IDs, timestamps, códigos); o comportamento linear do Radix Sort o torna vantajoso nesses cenários em comparação a algoritmos O(n log n).
- Sistemas que exigem previsibilidade de tempo: por ter desempenho determinístico independente da entrada, o Radix Sort é adequado a sistemas de tempo real ou embarcados em que o pior caso precisa ser

conhecido com antecedência, evitando o risco de degradação para $O(n^2)$ presente em algoritmos como o QuickSort.

- Construção de suffix arrays e processamento de strings: variantes do Radix Sort (como o MSD Radix Sort) são utilizadas em bioinformática e em algoritmos de compressão de texto para ordenar sufixos e strings de tamanho fixo de forma eficiente.
- Ordenação em GPU e processamento paralelo: como cada rodada de Counting Sort pode ser paralelizada por partição, o Radix Sort é um dos algoritmos de ordenação mais utilizados em implementações para GPU (por exemplo, em bibliotecas CUDA), sendo aplicado em renderização gráfica, simulações físicas e ordenação de grandes conjuntos de dados em paralelo.
- Ordenação de chaves de tamanho fixo do mundo real: CEPs, CPFs, códigos de barras e datas em formato numérico são exemplos de chaves com número fixo de dígitos, cenário ideal para o Radix Sort, já que d permanece pequeno e constante independentemente de n.

10. Vantagens e desvantagens

Vantagens

- Complexidade linear O(n) quando d e k são constantes, superando o limite teórico O(n log n) dos algoritmos comparativos.
- Comportamento determinístico: o tempo de execução não varia conforme a organização inicial dos dados, eliminando o risco de um pior caso inesperado.
- É estável: preserva a ordem relativa de elementos com chaves iguais, o que é essencial em ordenações multi-critério.
- Permite paralelismo natural, já que cada rodada de Counting Sort pode ser dividida por partições dos dados.

Desvantagens

- Não é in-place: requer memória auxiliar O(n + k) para o vetor de saída e o vetor de contagem, o que pode ser proibitivo em sistemas com memória muito restrita.
- Sofre com cache misses: o acesso não sequencial à memória durante a fase de distribuição em baldes reduz a vantagem teórica em hardware moderno, especialmente para valores pequenos de n.
- Funciona naturalmente apenas para inteiros não negativos; números negativos, ponto flutuante ou strings exigem transformações adicionais que aumentam a complexidade da implementação.
- É sensível à escolha da base k: bases maiores reduzem o número de rodadas, mas aumentam o tamanho do vetor de contagem e a pressão sobre o cache.

11. Conclusão

O que foi aprendido: o estudo do Radix Sort mostrou que é possível contornar a barreira teórica de O(n log n) imposta aos algoritmos comparativos ao explorar a estrutura interna dos dados — no caso, a representação posicional de números inteiros em dígitos. Ao substituir comparações diretas por distribuições sucessivas em baldes, o algoritmo alcança complexidade próxima de linear sem jamais comparar dois elementos entre si.

Principal vantagem: seu comportamento determinístico, aliado à complexidade $O(d \times (n + k))$ — que se aproxima de O(n) quando o número de dígitos é fixo — torna o tempo de execução previsível e independente da organização da entrada, algo que nenhum dos algoritmos comparativos estudados (Bubble Sort, QuickSort, MergeSort) garante simultaneamente.

Principal limitação: o algoritmo não é in-place, exigindo memória auxiliar O(n + k), e está naturalmente restrito a chaves inteiras ou de tamanho fixo, não sendo uma solução geral para qualquer tipo de dado sem adaptações.

Quando utilizar: o Radix Sort é a escolha adequada para grandes volumes de inteiros ou chaves de tamanho fixo (IDs, CEPs, timestamps), sistemas que exigem previsibilidade de tempo de execução e cenários em que o paralelismo entre rodadas pode ser explorado, como em GPU.

Quando não utilizar: não é recomendado quando a memória disponível é muito restrita (por exemplo, sistemas embarcados com poucos kilobytes de RAM), quando os dados não são numéricos de tamanho fixo sem conversão prévia, ou quando o volume de dados é pequeno, caso em que o overhead das rodadas de Counting Sort pode superar o benefício da complexidade linear.

Comparando com Bubble Sort, QuickSort e MergeSort: para 1 milhão de inteiros, a escolha recairia sobre o Radix Sort caso o objetivo fosse desempenho linear e previsível, especialmente se os valores tiverem um número limitado de dígitos e houver memória auxiliar disponível — nesse cenário, ele tende a superar o QuickSort e o MergeSort em tempo de execução. Entretanto, se a aplicação envolvesse tipos de dados heterogêneos, memória extremamente restrita ou a necessidade de um algoritmo de propósito geral, o QuickSort seria preferível pela menor constante de tempo e boa localidade de cache, ou o MergeSort, caso a estabilidade e o pior caso garantido de O(n log n) fossem requisitos obrigatórios. O Bubble Sort, em qualquer desses cenários de grande escala, seria descartado por seu comportamento quadrático.

12. Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009. Capítulo 8: Sorting in Linear Time — referência canônica para a análise formal de complexidade e a prova de corretude do Radix Sort.

KNUTH, D. E. The Art of Computer Programming, Volume 3: Sorting and Searching. 2. ed. Reading: Addison-Wesley, 1998. Análise histórica detalhada das máquinas de tabulação de Hollerith (1887) e da genealogia do Radix Sort.

SEDGEWICK, R.; WAYNE, K. Algorithms. 4. ed. Upper Saddle River: Addison-Wesley, 2011. Implementações comentadas de LSD e MSD Radix Sort, com benchmarks comparativos entre algoritmos de ordenação.

NVIDIA CORPORATION. CUDA C++ Programming Guide. Versão 12.x. Santa Clara: NVIDIA, 2023. Documentação oficial de implementação paralela de algoritmos de ordenação em GPUs, incluindo Radix Sort.

GORELICK, M.; OZSVALD, I. High Performance Python: Practical Performant Programming for Humans. 2. ed. Sebastopol: O'Reilly Media, 2020. Análise de cache misses, localidade de referência e impacto no desempenho prático de algoritmos de ordenação em hardware moderno.